

Chapter 6. Reading and Writing Simple Records

As mentioned in Chapter 5, many applications deal with data that is *persistent* - meaning that the data lives longer than the program by being stored on disk in files. You can shut down the program and open it back up, and you are back where you started. Now, there are two basic kinds of persistent data - structured and unstructured. Unstructured data is like what we dealt with in the *toupper* program. It just dealt with text files that were entered by a person. The contents of the files weren't usable by a program because a program can't interpret what the user is trying to say in random text.

Structured data, on the other hand, is what computers excel at handling. Structured data is data that is divided up into fields and records. For the most part, the fields and records are fixed-length. Because the data is divided into fixed-length records and fixed-format fields, the computer can interpret the data. Structured data can contain variable-length fields, but at that point you are usually better off with a database.¹

This chapter deals with reading and writing simple fixed-length records. Let's say we wanted to store some basic information about people we know. We could imagine the following example fixed-length record about people:

- Firstname - 40 bytes
- Lastname - 40 bytes
- Address - 240 bytes
- Age - 4 bytes

1. A database is a program which handles persistent structured data for you. You don't have to write the programs to read and write the data to disk, to do lookups, or even to do basic processing. It is a very high-level interface to structured data which, although it adds some overhead and additional complexity, is very useful for complex data processing tasks. References for learning how databases work are listed in Chapter 13.

Chapter 6. Reading and Writing Simple Records

In this, everything is character data except for the age, which is simply a numeric field, using a standard 4-byte word (we could just use a single byte for this, but keeping it at a word makes it easier to process).

In programming, you often have certain definitions that you will use over and over again within the program, or perhaps within several programs. It is good to separate these out into files that are simply included into the assembly language files as needed. For example, in our next programs we will need to access the different parts of the record above. This means we need to know the offsets of each field from the beginning of the record in order to access them using base pointer addressing. The following constants describe the offsets to the above structure. Put them in a file named `record-def.s`:

```
.equ RECORD_FIRSTNAME, 0
.equ RECORD_LASTNAME, 40
.equ RECORD_ADDRESS, 80
.equ RECORD_AGE, 320

.equ RECORD_SIZE, 324
```

In addition, there are several constants that we have been defining over and over in our programs, and it is useful to put them in a file, so that we don't have to keep entering them. Put the following constants in a file called `linux.s`:

```
#Common Linux Definitions

#System Call Numbers
.equ SYS_EXIT, 1
.equ SYS_READ, 3
.equ SYS_WRITE, 4
.equ SYS_OPEN, 5
.equ SYS_CLOSE, 6
.equ SYS_BRK, 45
```

```
#System Call Interrupt Number
.equ LINUX_SYSCALL, 0x80

#Standard File Descriptors
.equ STDIN, 0
.equ STDOUT, 1
.equ STDERR, 2

#Common Status Codes
.equ END_OF_FILE, 0
```

We will write three programs in this chapter using the structure defined in `record-def.s`. The first program will build a file containing several records as defined above. The second program will display the records in the file. The third program will add 1 year to the age of every record.

In addition to the standard constants we will be using throughout the programs, there are also two functions that we will be using in several of the programs - one which reads a record and one which writes a record.

What parameters do these functions need in order to operate? We basically need:

- The location of a buffer that we can read a record into
- The file descriptor that we want to read from or write to

Let's look at our reading function first:

```
.include "record-def.s"
.include "linux.s"

#PURPOSE:   This function reads a record from the file
```

Chapter 6. Reading and Writing Simple Records

```
#           descriptor
#
#INPUT:      The file descriptor and a buffer
#
#OUTPUT:     This function writes the data to the buffer
#            and returns a status code.
#
#STACK LOCAL VARIABLES
.equ ST_READ_BUFFER, 8
.equ ST_FILEDES, 12
.section .text
.globl read_record
.type read_record, @function
read_record:
    pushl %ebp
    movl %esp, %ebp

    pushl %ebx
    movl ST_FILEDES(%ebp), %ebx
    movl ST_READ_BUFFER(%ebp), %ecx
    movl $RECORD_SIZE, %edx
    movl $SYS_READ, %eax
    int $LINUX_SYSCALL

    #NOTE - %eax has the return value, which we will
    #       give back to our calling program
    popl %ebx

    movl %ebp, %esp
    popl %ebp
    ret
```

It's a pretty simply function. It just reads data the size of our structure into an appropriately sized buffer from the given file descriptor. The writing one is similar:

```
.include "linux.s"
#include "record-def.s"
#PURPOSE:   This function writes a record to
#           the given file descriptor
#
#INPUT:      The file descriptor and a buffer
#
#OUTPUT:     This function produces a status code
#
#STACK LOCAL VARIABLES
.equ ST_WRITE_BUFFER, 8
.equ ST_FILEDES, 12
.section .text
.globl write_record
.type write_record, @function
write_record:
    pushl %ebp
    movl %esp, %ebp

    pushl %ebx
    movl $SYS_WRITE, %eax
    movl ST_FILEDES(%ebp), %ebx
    movl ST_WRITE_BUFFER(%ebp), %ecx
    movl $RECORD_SIZE, %edx
    int $LINUX_SYSCALL

    #NOTE - %eax has the return value, which we will
    #       give back to our calling program
    popl %ebx

    movl %ebp, %esp
    popl %ebp
    ret
```

Chapter 6. Reading and Writing Simple Records

Now that we have our basic definitions down, we are ready to write our programs.

Writing Records

This program will simply write some hardcoded records to disk. It will:

- Open the file
- Write three records
- Close the file

Type the following code into a file called `write-records.s`:

```
.include "linux.s"
.include "record-def.s"

.section .data

#Constant data of the records we want to write
#Each text data item is padded to the proper
#length with null (i.e. 0) bytes.

#.rept is used to pad each item. .rept tells
#the assembler to repeat the section between
#.rept and .endr the number of times specified.
#This is used in this program to add extra null
#characters at the end of each field to fill
#it up
record1:
.ascii "Fredrick\0"
.rept 31 #Padding to 40 bytes
.byte 0
.endr

.ascii "Bartlett\0"
```

Chapter 6. Reading and Writing Simple Records

```
.rept 31 #Padding to 40 bytes
.byte 0
.endr

.ascii "4242 S Prairie\nTulsa, OK 55555\0"
.rept 209 #Padding to 240 bytes
.byte 0
.endr

.long 45

record2:
.ascii "Marilyn\0"
.rept 32 #Padding to 40 bytes
.byte 0
.endr

.ascii "Taylor\0"
.rept 33 #Padding to 40 bytes
.byte 0
.endr

.ascii "2224 S Johannan St\nChicago, IL 12345\0"
.rept 203 #Padding to 240 bytes
.byte 0
.endr

.long 29

record3:
.ascii "Derrick\0"
.rept 32 #Padding to 40 bytes
.byte 0
.endr
```

Chapter 6. Reading and Writing Simple Records

```
.ascii "McIntire\0"
.rept 31 #Padding to 40 bytes
.byte 0
.endr

.ascii "500 W Oakland\nSan Diego, CA 54321\0"
.rept 206 #Padding to 240 bytes
.byte 0
.endr

.long 36

#This is the name of the file we will write to
file_name:
.ascii "test.dat\0"

.equ ST_FILE_DESCRIPTOR, -4
.globl _start
_start:
#Copy the stack pointer to %ebp
movl %esp, %ebp
#Allocate space to hold the file descriptor
subl $4, %esp

#Open the file
movl $SYS_OPEN, %eax
movl $file_name, %ebx
movl $0101, %ecx #This says to create if it
                  #doesn't exist, and open for
                  #writing
movl $0666, %edx
int $LINUX_SYSCALL

#Store the file descriptor away
movl %eax, ST_FILE_DESCRIPTOR(%ebp)
```

```
#Write the first record
pushl ST_FILE_DESCRIPTOR(%ebp)
pushl $record1
call  write_record
addl  $8, %esp

#Write the second record
pushl ST_FILE_DESCRIPTOR(%ebp)
pushl $record2
call  write_record
addl  $8, %esp

#Write the third record
pushl ST_FILE_DESCRIPTOR(%ebp)
pushl $record3
call  write_record
addl  $8, %esp

#Close the file descriptor
movl  $SYS_CLOSE, %eax
movl  ST_FILE_DESCRIPTOR(%ebp), %ebx
int   $LINUX_SYSCALL

#Exit the program
movl  $SYS_EXIT, %eax
movl  $0, %ebx
int   $LINUX_SYSCALL
```

This is a fairly simple program. It merely consists of defining the data we want to write in the `.data` section, and then calling the right system calls and function calls to accomplish it. For a refresher of all of the system calls used, see Appendix C.

Chapter 6. Reading and Writing Simple Records

You may have noticed the lines:

```
.include "linux.s"  
.include "record-def.s"
```

These statements cause the given files to basically be pasted right there in the code. You don't need to do this with functions, because the linker can take care of combining functions exported with `.globl`. However, constants defined in another file do need to be imported in this way.

Also, you may have noticed the use of a new assembler directive, `.rept`. This directive repeats the contents of the file between the `.rept` and the `.endr` directives the number of times specified after `.rept`. This is usually used the way we used it - to pad values in the `.data` section. In our case, we are adding null characters to the end of each field until they are their defined lengths.

To build the application, run the commands:

```
as write-records.s -o write-record.o  
as write-record.s -o write-record.o  
ld write-record.o write-records.o -o write-records
```

Here we are assembling two files separately, and then combining them together using the linker. To run the program, just type the following:

```
./write-records
```

This will cause a file called `test.dat` to be created containing the records. However, since they contain non-printable characters (the null character, specifically), they may not be viewable by a text editor. Therefore we need the next program to read them for us.

Reading Records

Now we will consider the process of reading records. In this program, we will read each record and display the first name listed with each record.

Since each person's name is a different length, we will need a function to count the number of characters we want to write. Since we pad each field with null characters, we can simply count characters until we reach a null character.² Note that this means our records must contain at least one null character each.

Here is the code. Put it in a file called `count_chars.s`:

```
#PURPOSE:  Count the characters until a null byte is reached.
#
#INPUT:    The address of the character string
#
#OUTPUT:   Returns the count in %eax
#
#PROCESS:
#  Registers used:
#    %ecx - character count
#    %al  - current character
#    %edx - current character address

.type count_chars, @function
.globl count_chars

#This is where our one parameter is on the stack
.equ ST_STRING_START_ADDRESS, 8
count_chars:
    pushl %ebp
    movl  %esp, %ebp

    #Counter starts at zero
    movl  $0, %ecx
```

2. If you have used C, this is what the `strlen` function does.

Chapter 6. Reading and Writing Simple Records

```
#Starting address of data
movl  ST_STRING_START_ADDRESS(%ebp), %edx

count_loop_begin:
#Grab the current character
movb  (%edx), %al
#Is it null?
cmpb  $0, %al
#If yes, we're done
je    count_loop_end
#Otherwise, increment the counter and the pointer
incl  %ecx
incl  %edx
#Go back to the beginning of the loop
jmp   count_loop_begin

count_loop_end:
#We're done. Move the count into %eax
#and return.
movl  %ecx, %eax

popl  %ebp
ret
```

As you can see, it's a fairly straightforward function. It simply loops through the bytes, counting as it goes, until it hits a null character. Then it returns the count.

Our record-reading program will be fairly straightforward, too. It will do the following:

- Open the file
- Attempt to read a record

- If we are at the end of the file, exit
- Otherwise, count the characters of the first name
- Write the first name to STDOUT
- Write a newline to STDOUT
- Go back to read another record

To write this, we need one more simple function - a function to write out a newline to STDOUT. Put the following code into `write-newline.s`:

```
.include "linux.s"
.globl write_newline
.type write_newline, @function
.section .data
newline:
.ascii "\n"
.section .text
.equ ST_FILEDES, 8
write_newline:
    pushl %ebp
    movl  %esp, %ebp

    movl  $SYS_WRITE, %eax
    movl  ST_FILEDES(%ebp), %ebx
    movl  $newline, %ecx
    movl  $1, %edx
    int   $LINUX_SYSCALL
    movl  %ebp, %esp
    popl  %ebp
    ret
```

Chapter 6. Reading and Writing Simple Records

Now we are ready to write the main program. Here is the code to `read-records.s`:

```
.include "linux.s"
.include "record-def.s"

.section .data
file_name:
.ascii "test.dat\0"

.section .bss
.lcomm record_buffer, RECORD_SIZE

.section .text
#Main program
.globl _start
_start:
#These are the locations on the stack where
#we will store the input and output descriptors
#(FYI - we could have used memory addresses in
#a .data section instead)
.equ ST_INPUT_DESCRIPTOR, -4
.equ ST_OUTPUT_DESCRIPTOR, -8

#Copy the stack pointer to %ebp
movl %esp, %ebp
#Allocate space to hold the file descriptors
subl $8, %esp

#Open the file
movl $SYS_OPEN, %eax
movl $file_name, %ebx
movl $0, %ecx    #This says to open read-only
```

```
movl  $0666, %edx
int   $LINUX_SYSCALL

#Save file descriptor

movl  %eax, ST_INPUT_DESCRIPTOR(%ebp)

#Even though it's a constant, we are
#saving the output file descriptor in
#a local variable so that if we later
#decide that it isn't always going to
#be STDOUT, we can change it easily.
movl  $STDOUT, ST_OUTPUT_DESCRIPTOR(%ebp)

record_read_loop:
pushl  ST_INPUT_DESCRIPTOR(%ebp)
pushl  $record_buffer
call   read_record
addl   $8, %esp

#Returns the number of bytes read.
#If it isn't the same number we
#requested, then it's either an
#end-of-file, or an error, so we're
#quitting
cmpl   $RECORD_SIZE, %eax
jne    finished_reading

#Otherwise, print out the first name
#but first, we must know it's size
pushl  $RECORD_FIRSTNAME + record_buffer
call   count_chars
addl   $4, %esp
```

Chapter 6. Reading and Writing Simple Records

```
movl    %eax, %edx
movl    ST_OUTPUT_DESCRIPTOR(%ebp), %ebx
movl    $SYS_WRITE, %eax
movl    $RECORD_FIRSTNAME + record_buffer, %ecx
int     $LINUX_SYSCALL

pushl   ST_OUTPUT_DESCRIPTOR(%ebp)
call    write_newline
addl    $4, %esp

jmp     record_read_loop

finished_reading:
movl    $SYS_EXIT, %eax
movl    $0, %ebx
int     $LINUX_SYSCALL
```

To build this program, we need to assemble all of the parts and link them together:

```
as read-record.s -o read-record.o
as count-chars.s -o count-chars.o
as write-newline.s -o write-newline.o
as read-records.s -o read-records.o
ld read-record.o count-chars.o write-newline.o \
    read-records.o -o read-records
```

The backslash in the first line simply means that the command continues on the next line. You can run your program by doing `./read-records`.

As you can see, this program opens the file and then runs a loop of reading, checking for the end of file, and writing the firstname. The one construct that might be new is the line that says:

```
pushl   $RECORD_FIRSTNAME + record_buffer
```

It looks like we are combining and add instruction with a push instruction, but we are not. You see, both `RECORD_FIRSTNAME` and `record_buffer` are constants. The first is a direct constant, created through the use of a `.equ` directive, while the latter is defined automatically by the assembler through its use as a label (it's value being the address that the data that follows it will start at). Since they are both constants that the assembler knows, it is able to add them together while it is assembling your program, so the whole instruction is a single immediate-mode push of a single constant.

The `RECORD_FIRSTNAME` constant is the number of bytes after the beginning of a record before we hit the first name. `record_buffer` is the name of our buffer for holding records. Adding them together gets us the address of the first name member of the record stored in `record_buffer`.

Modifying the Records

In this section, we will write a program that:

- Opens an input and output file
- Reads records from the input
- Increments the age
- Writes the new record to the output file

Like most programs we've encountered recently, this program is pretty straightforward.³

```
.include "linux.s"
.include "record-def.s"
```

3. You will find that after learning the mechanics of programming, most programs are pretty straightforward once you know exactly what it is you want to do. Most of them initialize data, do some processing in a loop, and then clean everything up.

Chapter 6. Reading and Writing Simple Records

```
.section .data
input_file_name:
.ascii "test.dat\0"

output_file_name:
.ascii "testout.dat\0"

.section .bss
.lcomm record_buffer, RECORD_SIZE

#Stack offsets of local variables
.equ ST_INPUT_DESCRIPTOR, -4
.equ ST_OUTPUT_DESCRIPTOR, -8

.section .text
.globl _start
_start:
#Copy stack pointer and make room for local variables
movl %esp, %ebp
subl $8, %esp

#Open file for reading
movl $SYS_OPEN, %eax
movl $input_file_name, %ebx
movl $0, %ecx
movl $0666, %edx
int $LINUX_SYSCALL

movl %eax, ST_INPUT_DESCRIPTOR(%ebp)

#Open file for writing
movl $SYS_OPEN, %eax
movl $output_file_name, %ebx
movl $0101, %ecx
```

```
movl    $0666, %edx
int     $LINUX_SYSCALL

movl    %eax, ST_OUTPUT_DESCRIPTOR(%ebp)

loop_begin:
pushl   ST_INPUT_DESCRIPTOR(%ebp)
pushl   $record_buffer
call    read_record
addl    $8, %esp

#Returns the number of bytes read.
#If it isn't the same number we
#requested, then it's either an
#end-of-file, or an error, so we're
#quitting
cmpl    $RECORD_SIZE, %eax
jne     loop_end

#Increment the age
incl    record_buffer + RECORD_AGE

#Write the record out
pushl   ST_OUTPUT_DESCRIPTOR(%ebp)
pushl   $record_buffer
call    write_record
addl    $8, %esp

jmp     loop_begin

loop_end:
movl    $SYS_EXIT, %eax
movl    $0, %ebx
int     $LINUX_SYSCALL
```

Chapter 6. Reading and Writing Simple Records

You can type it in as `add-year.s`. To build it, type the following⁴:

```
as add-year.s -o add-year.o
ld add-year.o read-record.o write-record.o -o add-year
```

To run the program, just type in the following⁵:

```
./add-year
```

This will add a year to every record listed in `test.dat` and write the new records to the file `testout.dat`.

As you can see, writing fixed-length records is pretty simple. You only have to read in blocks of data to a buffer, process them, and write them back out. Unfortunately, this program doesn't write the new ages out to the screen so you can verify your program's effectiveness. This is because we won't get to displaying numbers until Chapter 8 and Chapter 10. After reading those you may want to come back and rewrite this program to display the numeric data that we are modifying.

Review

Know the Concepts

- What is a record?
- What is the advantage of fixed-length records over variable-length records?
- How do you include constants in multiple assembly source files?

4. This assumes that you have already built the object files `read-record.o` and `write-record.o` in the previous examples. If not, you will have to do so.

5. This is assuming you created the file in a previous run of `write-records`. If not, you need to run `write-records` first before running this program.

- Why might you want to split up a project into multiple source files?
- What does the instruction `incl record_buffer + RECORD_AGE` do? What addressing mode is it using? How many operands does the `incl` instructions have in this case? Which parts are being handled by the assembler and which parts are being handled when the program is run?

Use the Concepts

- Add another data member to the person structure defined in this chapter, and rewrite the reading and writing functions and programs to take them into account. Remember to reassemble and relink your files before running your programs.
- Create a program that uses a loop to write 30 identical records to a file.
- Create a program to find the largest age in the file and return that age as the status code of the program.
- Create a program to find the smallest age in the file and return that age as the status code of the program.

Going Further

- Rewrite the programs in this chapter to use command-line arguments to specify the filenames.
- Research the `lseek` system call. Rewrite the `add-year` program to open the source file for both reading and writing (use `$2` for the read/write mode), and write the modified records back to the same file they were read from.
- Research the various error codes that can be returned by the system calls made in these programs. Pick one to rewrite, and add code that checks `%eax` for error

Chapter 6. Reading and Writing Simple Records

conditions, and, if one is found, writes a message about it to `STDERR` and `exit`.

- Write a program that will add a single record to the file by reading the data from the keyboard. Remember, you will have to make sure that the data has at least one null character at the end, and you need to have a way for the user to indicate they are done typing. Because we have not gotten into characters to numbers conversion, you will not be able to read the age in from the keyboard, so you'll have to have a default age.
- Write a function called `compare-strings` that will compare two strings up to 5 characters. Then write a program that allows the user to enter 5 characters, and have the program return all records whose first name starts with those 5 characters.